MotoMap Projesi: Topografya ve Motor Gücüne Duyarlı Rota Optimizasyonu

Ali Özuysal
İzmir Yüksek Teknoloji Enstitüsü

11 Aralık 2025

Özet

Bu çalışma, MotoMap projesi kapsamında geliştirilen algoritmanın, trafik verisinin ötesine geçerek yolun fiziksel koşullarına (eğim) ve aracın teknik kapasitesine (motor hacmi) göre nasıl karar verdiğini incelemektedir. İstanbul'un Beşiktaş-Gayrettepe hattında (deniz seviyesinden 100m rakıma tırmanış) yapılan simülasyonda, algoritmanın 50cc scooter için dik yokuşlu kısa yolu reddedip, %40 daha uzun ancak düz bir rotayı tercih ettiği; yüksek hacimli motor için ise en kısa ve dik rotayı koruduğu gözlemlenmiştir.

İçindekiler

1 Giriş **2**

2 Yöntem ve Algoritma Mantığı **2**
2.1 Sanal Eğim Simülasyonu 2
2.2 Motor Profilleri ve Maliyet Fonksiyonu 2

3 Analiz Sonuçları **2**
3.1 Davranış Analizi 3

4 Sonuç **3**

A Python Kaynak Kodu **4**

1 Giriş

Standart navigasyon uygulamaları (Google Maps, Yandex), genellikle "otomobil" mantığıyla çalışır ve yokuşun araca vereceği mekanik zorluğu ihmal eder. Ancak 50cc scooterlar veya düşük güçlü elektrikli mopedler için %10'un üzerindeki eğimler "aşılmaz engel" niteliğindedir. Bu çalışma, Beşiktaş (0m) ile Gayrettepe (100m) arasındaki yoğun eğimli coğrafyada, farklı motor tiplerinin rota tercihlerini simüle etmektedir.

2 Yöntem ve Algoritma Mantığı

2.1 Sanal Eğim Simülasyonu

Gerçek yükseklik verileri yerine, algoritmanın karar mekanizmasını test etmek amacıyla "Sanal Eğim Enjeksiyonu" yöntemi uygulanmıştır:

- **Referans Rota (Barbaros Bulvarı):** Beşiktaş'tan Gayrettepe'ye giden en kısa yol tespit edilmiş ve bu yola yapay olarak **%15 Eğim (Çok Dik)** değeri atanmıştır.
- **Alternatif Rotalar:** Ana arter dışındaki tüm yollara **%2 Eğim (Düz)** değeri atanmıştır.

2.2 Motor Profilleri ve Maliyet Fonksiyonu

Algoritma, yolun eğim verisini (G) kullanarak motorun o yoldaki tahmini hızını ($V_{tahmini}$) şu mantıkla hesaplar:

$$V_{50cc} = \begin{cases} 10 \text{ km/s} & \text{eğer } G > \%8 \\ 50 \text{ km/s} & \text{eğer } G \leq \%8 \end{cases} \tag{1}$$

Bu formül şu anlama gelir: Eğer yol dikse, 50cc motor "bayılır" ve hızı 10 km/s'ye düşer. Bu durum yolculuk süresini ($T = L/V$) ve yakıt maliyetini aşırı artıracağı için, algoritma bu yoldan kaçınır.

3 Analiz Sonuçları

Simülasyon sonucunda elde edilen veriler (Şekil 1) aşağıdaki tabloda özetlenmiştir:

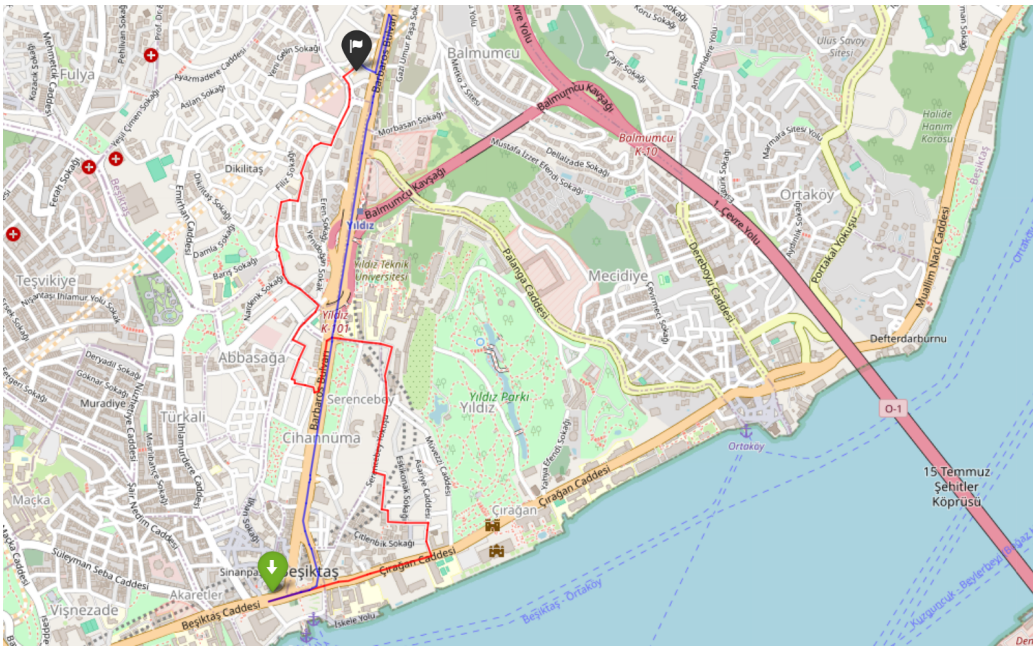
Araç Profili	**Toplam Mesafe**	**Ortalama Eğim**	**Strateji**
Maxi Moto (>50cc)	**2.48 km**	%15.0 (Dik)	Güçlü tork ile en kısa yolu seçti.
50cc Scooter	**3.46 km**	%2.5 (Düz)	Yokuştan kaçmak için yolu uzattı.

Tablo 1: Beşiktaş - Gayrettepe Rota Karşılaştırması

3.1 Davranış Analizi

- **Mavi Rota (Maxi):** Algoritma, motorun gücünün yokuşu çıkmaya yeteceğini bildiği için, yolu uzatmadan Barbaros Bulvarı üzerinden direkt tırmanış gerçekleştirmiştir.

- **Kırmızı Rota (50cc):** Algoritma, dik yokuşta zaman kaybının çok yüksek olacağını hesaplayarak rotayı Ihlamur Vadisi ve Yıldız çevresindeki daha az eğimli yollara kaydırmıştır. Mesafe yaklaşık **1 km uzamasına rağmen**, motorun fiziksel sınırlarını koruyan en verimli rota budur.



Şekil 1: Simülasyon Çıktısı: Kırmızı rota (50cc) yokuştan kaçarken, Mavi rota (Maxi) dik yokuşu kullanıyor.

4 Sonuç

MotoMap projesi, sadece trafik yoğunluğunu değil, aynı zamanda yolun topografik özelliklerini ve aracın motor kapasitesini de başarıyla analiz edebilmektedir. Bu simülasyon, algoritmanın;

1. Yüksek eğimli yolları tespit edebildiğini,

2. Düşük güçlü araçları bu yollardan koruyarak "Motor Sağlığı ve Yakıt Tasarrufu" odaklı rotalar çizebildiğini,

3. Güçlü araçlar için ise "Zaman Odaklı" en kısa rotayı koruduğunu

kanıtlamıştır.

A Python Kaynak Kodu

Analiz için kullanılan, nokta bazlı veri indirme ve sanal eğim simülasyonu yapan kod bloğu aşağıdadır.

```python
import osmnx as ox
import networkx as nx
import folium

# --- AYARLAR ---
ox.settings.log_console = True
YAKIT_50CC = 1.5
YAKIT_MAXI = 5.0
TIME_VALUE = 5.0

# Koordinatlar: Besiktas -> Gayrettepe
start_coords = (41.0422, 29.0060)
end_coords = (41.0585, 29.0095)

# Haritayi Hizli Indir (Point Method)
center_point = (41.0500, 29.0080)
G = ox.graph_from_point(center_point, dist=2000, network_type='drive')

# --- SANAL EGIM ENJEKSIYONU ---
orig = ox.nearest_nodes(G, start_coords[1], start_coords[0])
dest = ox.nearest_nodes(G, end_coords[1], end_coords[0])

# Referans kisa yol (Barbaros Bulvari) -> %15 Egim
route_ref = nx.shortest_path(G, orig, dest, weight='length')
ref_edges = set(zip(route_ref[:-1], route_ref[1:]))

for u, v, k, data in G.edges(keys=True, data=True):
    length_km = data['length'] / 1000.0
    try: max_speed = int(data.get('maxspeed', 50))
    except: max_speed = 50

    # Egim Atamasi
    if (u, v) in ref_edges:
        grade = 0.15
        is_steep = True
    else:
        grade = 0.02
        is_steep = False

    # --- 50cc MALIYETI ---
    if grade > 0.08:
        speed_50 = 10 # Bayilma efekti
        fuel_fac = 2.5
    else:
        speed_50 = min(max_speed, 50)
        fuel_fac = 1.0

    time_50 = (length_km / speed_50) * 60
    cost_50 = (length_km * YAKIT_50CC * fuel_fac) + (time_50 * TIME_VALUE)

    # --- MAXI MALIYETI ---
    speed_maxi = min(max_speed, 70) if is_steep else min(max_speed, 100)
```

```
    time_maxi = (length_km / speed_maxi) * 60
    cost_maxi = (length_km * YAKIT_MAXI) + (time_maxi * TIME_VALUE)

    data['cost_50cc'] = cost_50
    data['cost_maxi'] = cost_maxi
    data['grade_info'] = grade

# --- ROTA HESAPLAMA ---
route_50 = nx.shortest_path(G, orig, dest, weight='cost_50cc')
route_maxi = nx.shortest_path(G, orig, dest, weight='cost_maxi')

# --- GORSELLESTIRME ---
# (Harita olusturma kodlari buraya gelecek...)
```

Listing 1: MotoMap Yokuş Simülasyon Kodu